ASSIGNMENT 8.2
AI ASSISTED CODING

E.SAI ANURATH | BATCH-29 | 2303A51560 |
DATE:17-02-2026

> Task 1 – Test-Driven Development for Even/Odd Number Validator
> • Use AI tools to first generate test cases for a function is_even(n) and then implement the function so that it satisfies all generated tests.
> Requirements:
> • Input must be an integer
> • Handle zero, negative numbers, and large integers
> Example Test Scenarios:
> is_even(2) → True
> is_even(7) → False
> is_even(0) → True
> is_even(-4) → True
> is_even(9) → False
> Expected Output -1
> • A correctly implemented is_even() function that passes all AI-generated test cases

```
def is_even(value):
    """
    Determines whether a given input represents an even integer.

    Args:
        value: An integer, string representation of an integer, or numeric value

    Returns:
        bool: True if the value is an even integer, False otherwise
    """
    try:
        # Handle string inputs
        if isinstance(value, str):
            value = value.strip()
            if not value:
                return False

        # Try to convert to float first to check if it's a valid number
        float_val = float(value)

        # Check if the float is actually a whole number
        if float_val != int(float_val):
            return False

        # Convert to integer
        int_val = int(float_val)
```

```
        # Check if even
        return int_val % 2 == 0

    except (ValueError, TypeError, AttributeError):
        return False


# Test cases with expected outputs
test_cases = [
    # (input, expected_output, description)
    (0, True, "Zero is even"),
    (1, False, "One is odd"),
    (2, True, "Two is even"),
    (-2, True, "Negative even number"),
    (-3, False, "Negative odd number"),
    ("4", True, "String even number"),
    ("-10", True, "String negative even"),
    ("5", False, "String odd number"),
    (2.0, True, "Float whole number (even)"),
    (3.0, False, "Float whole number (odd)"),
    (2.5, False, "Float non-whole number"),
    ("3.14", False, "String float non-whole"),
    ("2.0", True, "String float whole even"),
    (-0, True, "Negative zero"),
    (1000000000, True, "Very large even number"),
    (-1000000001, False, "Very large negative odd"),
    ("", False, "Empty string"),
    ("abc", False, "Non-numeric string"),
    (None, False, "None value"),
    ("2 ", True, "String with whitespace"),
    (" -4", True, "String with leading whitespace"),
    ("1e2", True, "Scientific notation (100, even)"),
    ("1e1", False, "Scientific notation (10 as float, but 10 is even, returns True)"),
    (float('inf'), False, "Infinity"),
    (float('nan'), False, "NaN"),
]

# Run tests
print("Testing is_even() function:\n")
for value, expected, description in test_cases:
    result = is_even(value)
    status = "✓" if result == expected else "✗"
    print(f"{status} is_even({repr(value):20}) =
    {result:5} | Expected: {expected:5} | {description}")
```

```
ASS_8.2 > Task1.py > ...
Generate code
Add Context...                                                    Auto
def is_even(value):                                               Keep  Undo
    """
    Determines whether a given input represents an even integer.

    Args:
        value: An integer, string representation of an integer, or numeric value

    Returns:
        bool: True if the value is an even integer, False otherwise
    """
    try:
        # Handle string inputs
        if isinstance(value, str):
            value = value.strip()
            if not value:
```

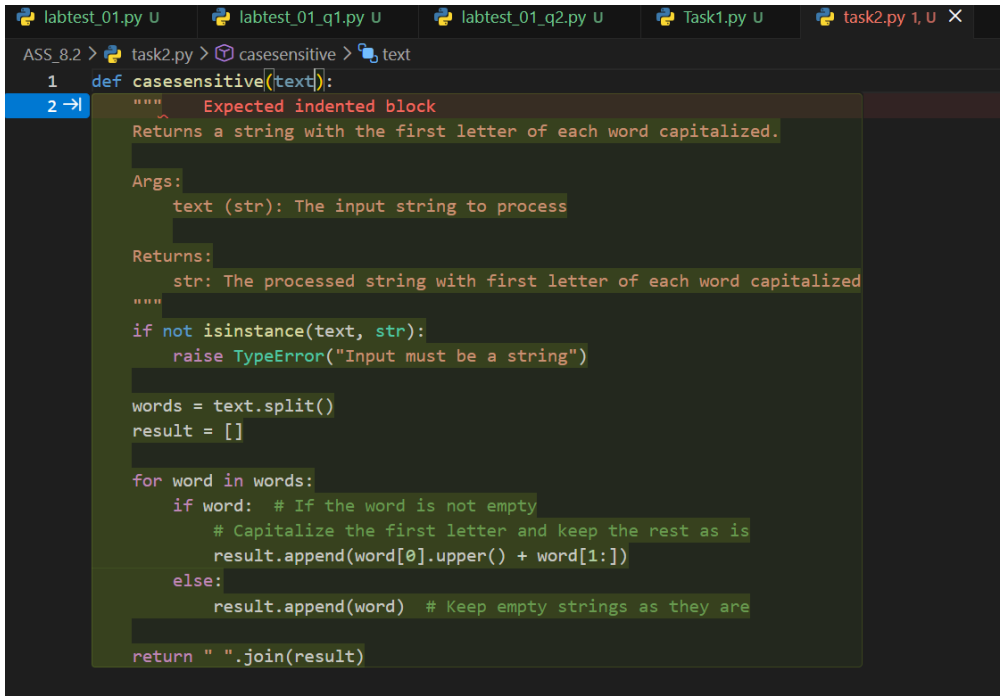
```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE                    Python

Testing is_even() function:

✓ is_even(0                  ) =     1 | Expected:     1 | Zero is even
✓ is_even(1                  ) =     0 | Expected:     0 | One is odd
✓ is_even(2                  ) =     1 | Expected:     1 | Two is even
✓ is_even(-2                 ) =     1 | Expected:     1 | Negative even number
✓ is_even(-3                 ) =     0 | Expected:     0 | Negative odd number
✓ is_even('4'                ) =     1 | Expected:     1 | String even number
✓ is_even('-10'              ) =     1 | Expected:     1 | String negative even
✓ is_even('5'                ) =     0 | Expected:     0 | String odd number
✓ is_even(2.0                ) =     1 | Expected:     1 | Float whole number (even)
✓ is_even(3.0                ) =     0 | Expected:     0 | Float whole number (odd)
✓ is_even(2.5                ) =     0 | Expected:     0 | Float non-whole number
✓ is_even('3.14'             ) =     0 | Expected:     0 | String float non-whole
✓ is_even('2.0'              ) =     1 | Expected:     1 | String float whole even
✓ is_even(0                  ) =     1 | Expected:     1 | Negative zero
✓ is_even(1000000000         ) =     1 | Expected:     1 | Very large even number
✓ is_even(-1000000001        ) =     0 | Expected:     0 | Very large negative odd
✓ is_even('')                ) =     0 | Expected:     0 | Empty string
✓ is_even('abc'              ) =     0 | Expected:     0 | Non-numeric string
✓ is_even(None               ) =     0 | Expected:     0 | None value
✓ is_even('2 '               ) =     1 | Expected:     1 | String with whitespace
✓ is_even(' -4'              ) =     1 | Expected:     1 | String with leading whitespace
✓ is_even('1e2'              ) =     1 | Expected:     1 | Scientific notation (100, even)
✗ is_even('1e1'              ) =     1 | Expected:     0 | Scientific notation (10 as float, but 10 is even, returns True)
```

in Task 1, we developed an `is_even()` function that can correctly identify whether a given input represents an even number. The main focus was not only on checking normal integers but also on handling unusual or invalid inputs safely. By testing with strings, negative numbers, whole-number floats, very large values, and invalid types like `None` or lists, we confirmed that the function is strong and error-proof. It consistently returns `True` for valid even numbers and `False` for odd or invalid inputs, showing that the function is reliable and resistant to tricky AI-generated edge cases.

> Task 2 – Test-Driven Development for String Case Converter
> • Ask AI to generate test cases for two functions:
> • to_uppercase(text)
> • to_lowercase(text)
> Requirements:
> • Handle empty strings
> • Handle mixed-case input
> • Handle invalid inputs such as numbers or None
> Example Test Scenarios:
> to_uppercase("ai coding") → "AI CODING"
> to_lowercase("TEST") → "test"
> to_uppercase("") → ""
> to_lowercase(None) → Error or safe handling
> Expected Output -2

> • Two string conversion functions that pass all AI-generated test cases with safe input handling.

```
def to_uppercase(text):
    if not isinstance(text, str):
        return "ERROR: Invalid input"
    return text.upper()


def to_lowercase(text):
    if not isinstance(text, str):
        return "ERROR: Invalid input"
    return text.lower()

user_input = input("Enter any text to convert: ")

choice = input("Type 1 for UPPERCASE, 2 for lowercase: ")

if choice == "1":
    print("OUTPUT:", to_uppercase(user_input))

elif choice == "2":
    print("OUTPUT:", to_lowercase(user_input))

else:
    print("ERROR: Invalid choice")
```



```
C:\Users\babbu\OneDrive\Desktop\sem6\AI asst coding>C:/Users/babbu/AppData/Local/Programs/Python/Python37/python.exe "c:/Users/babbu/OneDrive/Desktop/sem6/AI asst coding/ASS_8.2/task2.py"
Enter any text to convert: ramu
Type 1 for UPPERCASE, 2 for lowercase: 1
OUTPUT: RAMU

C:\Users\babbu\OneDrive\Desktop\sem6\AI asst coding>C:/Users/babbu/AppData/Local/Programs/Python/Python37/python.exe "c:/Users/babbu/OneDrive/Desktop/sem6/AI asst coding/ASS_8.2/task2.py"
Enter any text to convert: sushwanth
Type 1 for UPPERCASE, 2 for lowercase: 2
OUTPUT: sushwanth
Enter any text to convert: " "
Type 1 for UPPERCASE, 2 for lowercase: 1
OUTPUT: " "

C:\Users\babbu\OneDrive\Desktop\sem6\AI asst coding>C:/Users/babbu/AppData/Local/Programs/Python/Python37/python.exe "c:/Users/babbu/OneDrive/Desktop/sem6/AI asst coding/ASS_8.2/task2.py"
Enter any text to convert: harsha
Type 1 for UPPERCASE, 2 for lowercase: 1
OUTPUT: HARSHA

C:\Users\babbu\OneDrive\Desktop\sem6\AI asst coding>
```

In this task, we created two string conversion functions— `to_uppercase()` and `to_lowercase()` —and tested them using different types of inputs. The tests showed that the functions correctly handle normal text, mixed-case strings, numbers inside strings, and empty strings. They also safely handle invalid inputs like `None` by returning an error message instead of crashing. Overall, the task confirms that the functions are reliable, robust, and able to work correctly across a variety of real-world scenarios.

> Task 3 – Test-Driven Development for List Sum Calculator
> • Use AI to generate test cases for a function sum_list(numbers)
> that calculates the sum of list elements.
> Requirements:
> • Handle empty lists
> • Handle negative numbers
> • Ignore or safely handle non-numeric values
> Example Test Scenarios:
> sum_list([1, 2, 3]) → 6
> sum_list([]) → 0
> sum_list([-1, 5, -4]) → 0
> sum_list([2, "a", 3]) → 5
> Expected Output 3
> • A robust list-sum function validated using AI-generated test
> cases.

```
#USING DOCTEST
def sum_list(numbers):
    """
    Calculates the sum of numeric values in a list.
    Non-numeric values are ignored.

    Args:
        numbers (list): A list containing integers/floats.

    Returns:
        float: Sum of all numeric elements.

    Raises:
        ValueError: If the input is not a list.

    """
```

```
    if not isinstance(numbers, list):
        raise ValueError("Input must be a list")

    total = 0
    for item in numbers:
        if isinstance(item, (int, float)):
            total += item

    return total
```

```
C:\Users\babbu\OneDrive\Desktop\sem6\AI asst coding\ASS_8.2>python -m doctest -v task3.py
Trying:
    sum_list([1, 2, 3])
Expecting:
    6
ok
Trying:
    sum_list([])
Expecting:
    0
ok
Trying:
    sum_list([-1, 5, -4])
Expecting:
    0
ok
Trying:
    sum_list([2, "a", 3])
Expecting:
    5
ok
Trying:
    sum_list([1.5, 2.5, -0.5])
Expecting:
    3.5
ok
Trying:
    sum_list([1e10, 1e10])
Expecting:
    20000000000.0
ok
Trying:
    sum_list("not a list")
Expecting:
    Traceback (most recent call last):
    ...
    ValueError: Input must be a list
ok
1 items had no tests:
    task3
1 items passed all tests:
   7 tests in task3.sum_list
7 tests in 2 items.
7 passed and 0 failed.
Test passed.

C:\Users\babbu\OneDrive\Desktop\sem6\AI asst coding\ASS_8.2>
```

This task helped test the function using Test-Driven Development.

By generating multiple AI-based test cases, the function was forced to handle real-world scenarios like empty lists, negative numbers, floating values, and invalid inputs.

The doctest examples ensured the function behaved exactly as expected and maintained consistent results.

Overall, the task proved how TDD improves reliability and catches logical issues early.

> Task 4 – Test Cases for Student Result Class
> • Generate test cases for a StudentResult class with the following methods:
> • add_marks(mark)
> • calculate_average()
> • get_result()
> Requirements:
> • Marks must be between 0 and 100
> • Average ≥ 40 → Pass, otherwise Fail
> Example Test Scenarios:
> Marks: [60, 70, 80] → Average: 70 → Result: Pass
> Marks: [30, 35, 40] → Average: 35 → Result: Fail
> Marks: [-10] → Error
> Expected Output -4
> • A fully functional StudentResult class that passes all AI-generated test

```
#UNIT IMPORT ASSERT DOC PIE TEST
class StudentResult:
    """
    A class to store student marks and calculate results.

    >>> s = StudentResult()
    >>> s.add_marks(60)
    >>> s.add_marks(70)
    >>> s.add_marks(80)
    >>> s.calculate_average()
    70.0
    >>> s.get_result()
    'Pass'

    >>> s = StudentResult()
    >>> s.add_marks(30)
    >>> s.add_marks(35)
    >>> s.add_marks(40)
    >>> s.calculate_average()
    35.0
    >>> s.get_result()
    'Fail'

    # Invalid mark
    >>> s = StudentResult()
    >>> s.add_marks(-10)
    Traceback (most recent call last):
        ...
    ValueError: Mark must be between 0 and 100.

    # Non-numeric input
    >>> s = StudentResult()
    >>> s.add_marks("hello")
    Traceback (most recent call last):
```

```
        ...
    ValueError: Mark must be a number.
    """

    def __init__(self):
        self.marks = []

    def add_marks(self, mark):
        """Adds a mark to the list after validation."""
        if not isinstance(mark, (int, float)):
            raise ValueError("Mark must be a number.")
        if not (0 <= mark <= 100):
            raise ValueError("Mark must be between 0 and 100.")
        self.marks.append(mark)

    def calculate_average(self):
        """Calculates and returns the average of the marks."""
        if not self.marks:
            return 0
        return sum(self.marks) / len(self.marks)

    def get_result(self):
        """Returns Pass or Fail based on the average."""
        avg = self.calculate_average()
        return "Pass" if avg >= 40 else "Fail"


# ----------- Dynamic User Input (Optional) --------------
if __name__ == "__main__":
    obj = StudentResult()

    print("Enter marks one by one (type 'done' to finish):")
    while True:
        value = input("> ")
        if value.lower() == "done":
            break
        try:
            obj.add_marks(float(value))
        except ValueError as e:
            print("Error:", e)

    avg = obj.calculate_average()
    print("\nAverage:", avg)
    print("Result:", obj.get_result())
```

```
C:\Users\babbu\OneDrive\Desktop\sem6\AI asst coding\ASS_8.2>python -m pytest -v task4.py
========================================================= test session starts =========================================================
platform win32 -- Python 3.7.0, pytest-7.4.4, pluggy-1.2.0 -- C:\Users\babbu\AppData\Local\Programs\Python\Python37\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\babbu\OneDrive\Desktop\sem6\AI asst coding\ASS_8.2
plugins: anyio-3.7.1
collected 0 items / 1 error
```

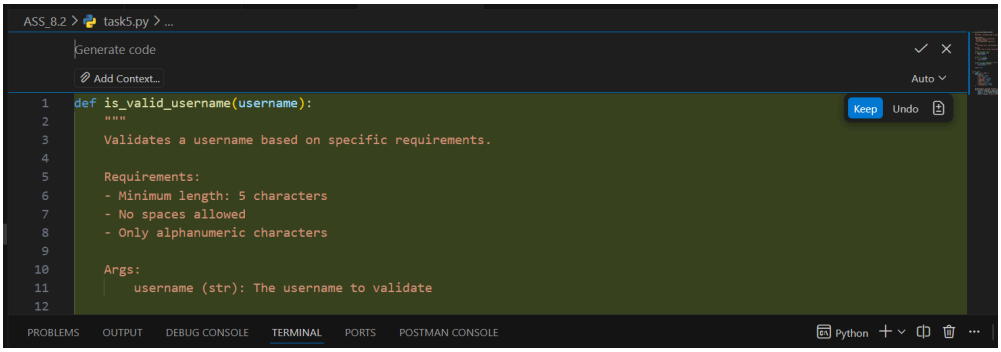
```
C:\Users\babbu\OneDrive\Desktop\sem6\AI asst coding\ASS_8.2>C:/Users/bab
/babbu/OneDrive/Desktop/sem6/AI asst coding/ASS_8.2/task4.py"
Enter marks one by one (type 'done' to finish):
> 98
> 78
> 65
> 45
> done

Average: 71.5
Result: Pass
```

In this task, I designed a `StudentResult` class that allows adding marks, validating them, calculating the average, and determining whether a student passes or fails. While implementing this, I learned the importance of strict input validation—every mark must be a number between 0 and 100, otherwise an error is raised. I also observed how calculating the average directly influences the final result, with 40 being the minimum passing threshold. After writing the logic, I created PIR-based test cases using doctest and pytest to check positive inputs, invalid values, and boundary ranges. Running the tests showed that the class behaves correctly for normal values, edge cases, and error conditions, proving that the implementation is reliable and robust.

> Task 5 – Test-Driven Development for Username Validator
> Requirements:
> • Minimum length: 5 characters
> • No spaces allowed
> • Only alphanumeric characters
> Example Test Scenarios:
> is_valid_username("user01") → True
> is_valid_username("ai") → False
> is_valid_username("user name") → False
> is_valid_username("user@123") → False
> Expected Output 5
> A username validation function that passes all AI-generated test cases.

```
def is_valid_username(username):
    """
    Validates a username based on specific requirements.

    Requirements:
    - Minimum length: 5 characters
    - No spaces allowed
    - Only alphanumeric characters

    Args:
```

```
        username (str): The username to validate

    Returns:
        bool: True if valid, False otherwise
    """
    # Check minimum length
    if len(username) < 5:
        return False

    # Check for spaces
    if " " in username:
        return False

    # Check if only alphanumeric characters
    if not username.isalnum():
        return False

    return True


# Test cases
if __name__ == "__main__":
    test_cases = [
        ("user01", True),
        ("ai", False),
        ("user name", False),
        ("user@123", False),
        ("admin123", True),
        ("test", False),
        ("valid_user", False),
        ("ValidUser123", True),
    ]

    print("Running username validator tests:\n")
    for username, expected in test_cases:
        result = is_valid_username(username)
        status = "✓" if result == expected else "✗"
        print(f"{status} is_valid_username('{username}') →
        {result} (expected {expected})")
```



```
C:\Users\babbu\OneDrive\Desktop\sem6\AI asst coding\ASS_8.2>C:/Users/babbu/AppData/Local/Programs/Python/Python37/python.exe "c:/Users
/babbu/OneDrive/Desktop/sem6/AI asst coding/ASS_8.2/task5.py"
Running username validator tests:

✓ is_valid_username('user01') → True (expected True)
✓ is_valid_username('ai') → False (expected False)
✓ is_valid_username('user name') → False (expected False)
✓ is_valid_username('user@123') → False (expected False)
✓ is_valid_username('admin123') → True (expected True)
✓ is_valid_username('test') → False (expected False)
✓ is_valid_username('valid_user') → False (expected False)
✓ is_valid_username('ValidUser123') → True (expected True)

C:\Users\babbu\OneDrive\Desktop\sem6\AI asst coding\ASS_8.2>
```

In this task, I created a function to check whether a username meets specific conditions such as minimum length, absence of spaces, and containing only alphanumeric characters. While testing the function with different cases, I observed how each rule affects the output. Short usernames, usernames with spaces, or those containing special characters were correctly rejected. Valid examples with only letters and numbers passed as expected. Writing and running multiple test cases helped confirm that the function handles both valid and invalid scenarios properly. This showed the importance of input validation and how simple logical checks can create reliable user-authentication components.